

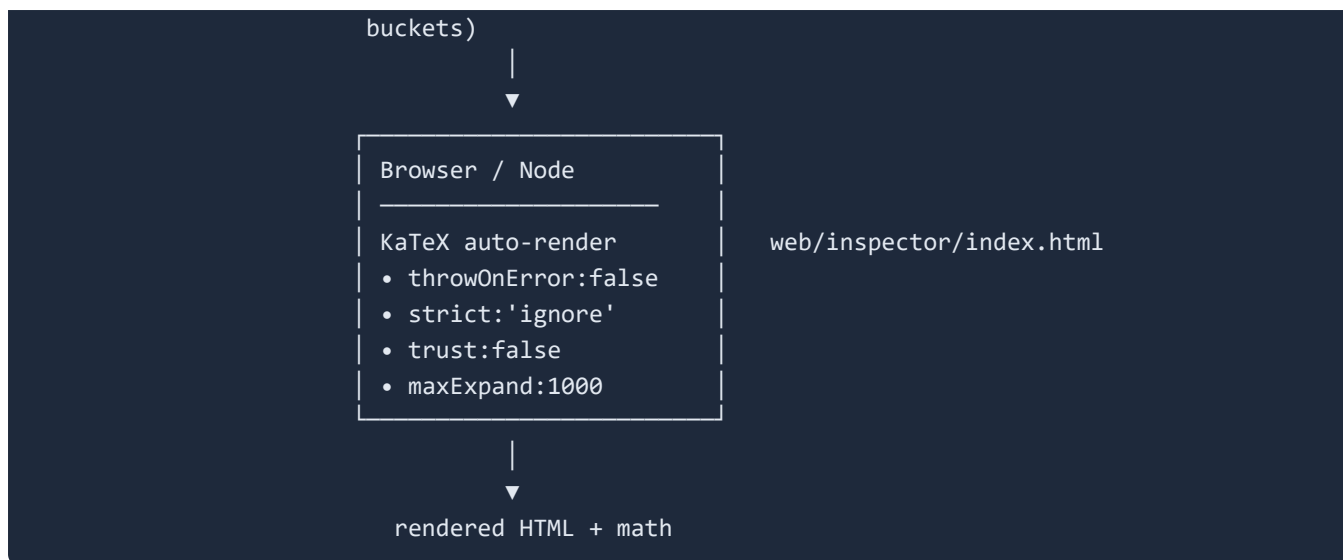
End-to-End Workflow — Complete Implementation Reference

Full lifecycle of the LaTeX auto-fix pipeline: every stage, every bucket, every repair, every render path, with the file each piece lives in.

1. High-level workflow







2. Bucket reference (every failure shape we recognize)

Each input is labeled with one or more bucket codes by `BucketLabeler` (`bucket_labeler.py`). A row can fire multiple buckets simultaneously.

Code	Name	What triggers it	Auto-fix	KaTeX error if unfixed
A	missing_delimiters	<code>\alpha + \beta</code> = ... (raw command without <code>\$. \$</code>)	<code>wrap_math_only</code>	silent — renders as plain text in browser auto-render mode
B	broken_braces	Unbalanced <code>{...}</code> or <code>...}</code>	<code>MissingFracPrefixRepairer</code> for <code>X}{Y}</code> ; else fallback	Expected <code>'</code> ' at end of input, or Expected <code>'EOF'</code> , got <code>'</code> '
C	unbalanced_dollar	Odd count of unescaped <code>\$</code>	<code>wrap_math_only_strip_orphan_dollar</code> OR segmenter truncation guard	Can't use function <code>'\$'</code> in math mode
D	ocr_corruption	Orphan <code>\eta</code> , <code>\rac</code> , <code>\ext</code> , <code>\alphaeta</code> , combining diacritics	<code>repair_orphan_eta_to_beta</code> , <code>repair_orphan_rac</code> , <code>repair_orphan_ext</code> , <code>repair_alphaeta</code> , <code>repair_orphan_greek</code> , <code>strip_combining_diacritics</code>	usually silent (wrong render); diacritic = Expected <code>'EOF'</code> , got <code>'<char>'</code>
E	invalid_command_name	<code>\input</code> , <code>\write</code> , <code>\href</code> , etc. (security-blocked)	none — fallback	Undefined control sequence: <code>\<cmd></code>
F	unsupported_environment	<code><table></code> , <code><p></code> , <code>
</code> etc.	HTML-aware split: tags passthrough, math inside still renders	No such environment: <code><name></code> or Unsupported character
G	mixed_prose_math	Normal prose with <code>\$. \$</code> math (control bucket — usually valid)	none needed	n/a
H	false_positive	_____ fill-blanks, single Greek letters in prose	<code>FillBlankInTextRepairer</code> converts to <code>\rule</code> ; else skip math	Expected group after <code>'_'</code>
I	code_snippet	Triple-backtick fences, <code><pre></code> , inline <code>`code`</code>	route to fallback (HTML-render path); no math parsing	n/a
J	currency_text	<code>\$5</code> , <code>\$10 each</code> , <code>Cost: \$100 USD</code> in prose	none — pipeline correctly leaves alone	Can't use function <code>'\$'</code> in math mode (if treated as math)
K	nested_parser_corruption	<code>\frac{1}{\text{\sqrt{3}}}</code> (math inside text inside math)	passthrough — KaTeX <code>strict='ignore'</code> handles	Can't use function <code>'\sqrt'</code> in text mode
L	multiline_aisolution	Literal <code>\n</code> from JSON serialization round-trip	<code>literal_escape_to_whitespace</code>	Undefined control sequence: <code>\n<word></code>

Code	Name	What triggers it	Auto-fix	KaTeX error if unfixed
Z	clean	No issues detected	none	n/a

3. Repair reference (every named repair the pipeline can apply)

`pipeline_result.repairs_applied` is a list of these names. Every repair is **pure**, **idempotent**, and **render-time only** (never mutates the DB).

Repair name	File	Tier	Scope	What it does	Example
nfc_normalize	tier1_repairs.py	1	global	Unicode NFC normalize	combining-form chars collapse
strip_combining_diacritics	tier1_repairs.py	1	math segment	Remove U+0300..U+036F	$\alpha + \beta \rightarrow \alpha + \beta$
literal_escape_to_whitespace	tier1_repairs.py	1	prose	<code>\n / \t / \r</code> literal $\rightarrow$ real whitespace	Step 1. Step 2. $\rightarrow$ Step 1.<NL>Step 2.
wrap_math_only	tier2_repairs.py	2	global	Wrap math-only content in <code>\$. \$</code>	$\frac{1}{2} \rightarrow \frac{1}{2}$
wrap_math_only_strip_orphan_dollar	tier2_repairs.py	2	global	Strip orphan trailing/leading <code>\$</code> , then wrap	$\alpha = 1\$ \rightarrow \alpha = 1$
repair_orphan_eta_to_beta	tier2_repairs.py	2	math	<code>eta</code> (no <code>\</code>) $\rightarrow$ <code>\beta</code>	$\alpha + \text{eta} \rightarrow \alpha + \beta$
repair_orphan_rac	tier2_repairs.py	2	math	<code>rac{a}{b}</code> $\rightarrow$ <code>\frac{a}{b}</code>	$\text{rac}{1}{2} \rightarrow \frac{1}{2}$
repair_orphan_ext	tier2_repairs.py	2	math	<code>ext{X}</code> $\rightarrow$ <code>\text{X}</code>	$\text{ext}{red} \rightarrow \text{red}$
repair_alphaeta	tier2_repairs.py	2	math	<code>alphaeta</code> (glued) $\rightarrow$ <code>\alpha\beta</code>	$\alpha\eta \rightarrow \alpha\beta$
repair_orphan_greek	tier2_repairs.py	2	math	bare <code>beta / gamma</code> /etc $\rightarrow$ <code>\beta / \gamma</code> (context-checked)	$+ \gamma + \rightarrow + \gamma +$
repair_fill_blank_in_text	tier2_repairs.py	2	math	<code>\text{_____}</code> $\rightarrow$ <code>\text{\rule{Nem}{0.4pt}}</code>	$\text{_____} \rightarrow \text{\rule{6em}{0.4pt}}$
repair_missing_frac_prefix	tier2_repairs.py	2	math	<code>X}{Y}</code> with extra <code>}</code> $\rightarrow$ <code>\frac{X}{Y}</code>	$1}{2} \rightarrow \frac{1}{2}$

Tier 1 runs unconditionally. **Tier 2** runs only when classifier confidence ≥ 0.7 OR a corruption signal is detected.

4. Render-path reference (what the output HTML looks like)

Outcome	Segment kind	HTML emitted	Rendered by KaTeX?
MATH	<code>math_inline</code> , <code>math_display</code> , <code>math_paren</code> , <code>math_bracket</code>	<code>\$...\$</code>	✅ yes (KaTeX auto-render walks <code>.latex-math</code>)
TEXT	<code>text</code>	escaped string, <code>\n</code> → <code>
</code>	❌ no (plain DOM)
HTML	<code>html</code>	the original HTML tag verbatim (e.g., <code><table></code> , <code></tr></code>)	❌ no (rendered by browser HTML engine)
FALLBACK	any	<code>...</code>	❌ no (escaped raw content, never re-parsed)

The browser's KaTeX auto-render-min.js script picks up `<span class="latex-math">` elements and replaces their content with rendered math. Everything else is plain DOM and renders natively.

5. Component file map

```

renderer/
├── src/latex_pipeline/
│   ├── core/
│   │   ├── interfaces.py    Protocols (ISegmenter, IClassifier, IRepairer,
│   │   │                   IValidator, IFallbackRenderer, IBucketLabeler,
│   │   │                   IFamilyResolver, ISignalDetector)
│   │   ├── models.py        Dataclasses (Segment, SegmentResult, PipelineResult,
│   │   │                   ClassificationResult, ValidationResult, RepairOutcome,
│   │   │                   RenderOutcome enum, SegmentKind enum)
│   │   └── exceptions.py     LatexPipelineError, InvalidSegmentError
│   ├── segmentation/
│   │   └── state_machine_segementer.py  Splits text into Segments. Handles
│   │   │                               $..$, $$..$$, \(..\), \[..], escaped \$,
│   │   │                               odd-dollar truncation guard, empty-math
│   │   │                               coalescing.
│   ├── classification/
│   │   ├── signals.py        8 ISignalDetector implementations + MATH_COMMANDS
│   │   │                   (251 vetted KaTeX commands)
│   │   ├── math_intent_classifier.py  Composes detectors → score + flags
│   │   └── family_resolver.py  field_path → source family + math prior
│   │   │                   (rubric_criterion: 0.85, ai_solution: 0.25, ...)
│   ├── repair/
│   │   ├── tier1_repairs.py    NfcNormalizer, DiacriticStripper, LiteralEscapeRepairer
│   │   └── tier2_repairs.py    MathOnlyWrapper, OrphanBackslashRepairer,
│   │   │                   FillBlankInTextRepairer, MissingFracPrefixRepairer
│   ├── validation/
│   │   └── validators.py       NonEmpty, MaxLength, BraceBalance, SubscriptRun,
│   │   │                   ForbiddenCommand, CompositeValidator
│   ├── buckets/
│   │   └── bucket_labeler.py   Maps result → bucket codes A..L, Z
│   ├── fallback/
│   │   └── html_fallback.py     HtmlFallbackRenderer (HTML-escape + label span)
│   ├── diagnostics/
│   │   └── error_analyzer.py    Pre-parse "missing character" pinpointer
│   │   │                   (with caret marker in the snippet)
│   └── pipeline/
│       ├── pipeline.py         Pipeline.run() orchestrator
│       │   ├── HTML-aware path
│       │   ├── Tier 1 global + Tier 2 global
│       │   ├── Segment → per-segment process
│       │   ├── Per-segment Tier 1 + Tier 2 + validate
│       │   └── Bucket label
│       └── builder.py           PipelineBuilder fluent DI builder
│   │   │                   + build_default_pipeline() factory

```

└─ app/	CLIs
└─ autoheal.py	Heal a single string (--diagnose, --interactive)
└─ run_pipeline.py	Heal a CSV/JSONL dataset
└─ build_inspector.py	Generate data.json for the observability UI
└─ web/	Browser-side assets
└─ renderer.js	JS port of the pipeline (matches Python output)
└─ playground.html	Live editor + KaTeX render
└─ inspector/	Observability UI
└─ index.html	Dashboard + filters + row cards
└─ styles.css	UI styling
└─ app.js	Interactivity (state, filters, expand, live heal)
└─ data.json	Generated by build_inspector.py
└─ tests_solid/	60-test SOLID test suite
└─ test_correct_latex.py	15 tests on valid LaTeX inputs
└─ test_broken_latex.py	21 tests on broken inputs (every bucket)
└─ test_solid_principles.py	12 tests for OCP/LSP/DIP compliance
└─ test_error_analyzer.py	11 tests for the pre-parse diagnostic
└─ tests/	28-test back-compat test suite
└─ docs/	Reference documentation
└─ KATEX_PARSE_ERRORS.md	Full catalog of 75 KaTeX ParseError templates
└─ katex_error_catalog.json	Machine-readable error catalog
└─ katex_commands_missing.txt	KaTeX commands we don't track as math-intent
└─ END_TO_END_WORKFLOW.md	This document
└─ out_solid/	CLI output (per dataset slice)
└─ all_candidates/	results.jsonl + report.md/json
└─ full/	(classified_candidates.jsonl)
└─ confirmed_broken/	
└─ ... etc	
└─ fallback_rows.jsonl	Per-row fallback inspection (0 rows now)

6. End-to-end examples per bucket

Each example shows the full lifecycle: **raw input** → **buckets** → **repairs** → **prepared output** → **render outcome**.

Bucket A — missing_delimiters

```

INPUT  : \alpha + \beta = -\frac{1}{6}
FAMILY : rubric_criterion (prior 0.85)
STAGE 2: MathOnlyWrapper detects math-only content with \alpha + \beta + \frac
  ✓ wrap_math_only fires
STAGE 3: Segmenter sees $\alpha + \beta = -\frac{1}{6}$, emits 1 math_inline
STAGE 4: Tier 1 math + classifier (score=1.0, math)
STAGE 5: Validator OK
BUCKETS: [A]
OUTPUT : $\alpha + \beta = -\frac{1}{6}$ ← KaTeX renders  $\alpha + \beta = -\frac{1}{6}$ 

```

Bucket B — broken_braces (auto-recovered via Case 4 fix)

```
INPUT   : 2(2)\sqrt{3} - 1}{2+\sqrt{3}}
FAMILY  : feedback (prior 0.25)
STAGE 3: Segmenter sees raw text (no math delim) – but Tier 2 detects pattern
STAGE 4: MissingFracPrefixRepairer sees X}{Y} + 1 extra }
  ✓ repair_missing_frac_prefix fires → wraps as \frac{X}{Y}
BUCKETS: [B, D]
OUTPUT  : $\frac{2(2)\sqrt{3} - 1}{2+\sqrt{3}}$
```

Bucket C — unbalanced_dollar

```
INPUT   : \alpha + eta = -\frac{1}{6}$      ← orphan trailing $
FAMILY  : rubric_criterion
STAGE 2: 1 unescaped $, orphan at end of math-shaped content
  ✓ wrap_math_only_strip_orphan_dollar → strips, wraps
STAGE 5: orphan-eta repair fires inside math
  ✓ repair_orphan_eta_to_beta
BUCKETS: [A, D]
OUTPUT  : $\alpha + \beta = -\frac{1}{6}$
```

Bucket D — ocr_corruption (multiple variants)

```
INPUT   : $\alpha + eta + \alpha\beta$
STAGE 5: OrphanBackslashRepairer detects 'eta' adjacent to math context
  ✓ repair_orphan_eta_to_beta
OUTPUT  : $\alpha + \beta + \alpha\beta$

INPUT   : $\frac{1}{3} \times rac{2}{5}$
  ✓ repair_orphan_rac
OUTPUT  : $\frac{1}{3} \times \frac{2}{5}$

INPUT   : $\alpha + \alpha\beta = 5$
  ✓ repair_alphaeta
OUTPUT  : $\alpha + \alpha\beta = 5$

INPUT   : $\alpha + \beta + \alpha\beta$      (with U+0311)
  ✓ strip_combining_diacritics
OUTPUT  : $\alpha + \beta + \alpha\beta$
```

Bucket E — invalid_command_name (security)

```
INPUT   : $\input{/etc/passwd}$
STAGE 5: ForbiddenCommandValidator blocks
BUCKETS: [E]
OUTPUT  : <span class="latex-fallback" data-render-reason="forbidden_command:input">
  $\input{/etc/passwd}$</span>
  (rendered as escaped text – never executed)
```

Bucket F — unsupported_environment (HTML-aware path)

```
INPUT  : Find the data: <table><tr><td>$x^2$</td></tr></table> when $y > 0$
ROUTE  : _run_html_aware (HTML detected)
SPLIT  : prose:"Find the data: "
         html : "<table>", "<tr>", "<td>"
         math  : "$x^2$" ← still renders!
         html  : "</td>", "</tr>", "</table>"
         prose : " when "
         math  : "$y > 0$" ← still renders!
BUCKETS: [F, G]
OUTPUT : Find the data: <table><tr><td><span class="latex-math">$x^2$</span></td></tr>
</table>
        when <span class="latex-math">$y > 0$</span>
        (browser renders the real <table>; KaTeX renders both math spans)
```

Bucket G — mixed_prose_math (control / clean)

```
INPUT  : If $\alpha$ and $\beta$ are zeroes of $x^2 - 1$, find $\alpha+\beta$.
STAGE 3: Segmenter produces 9 segments alternating text/math
STAGE 5: All math segments validate OK
BUCKETS: [G]
OUTPUT : (unchanged – exactly matches input)
```

Bucket H — false_positive (fill-in-the-blank fix)

```
INPUT  : x = \text{_____}
STAGE 5: SubscriptRunValidator triggers on _{3,} but
        FillBlankInTextRepairer rewrites the _____ first
        ✓ repair_fill_blank_in_text
OUTPUT : $x = \text{\rule{6em}{0.4pt}}$ ← KaTeX renders horizontal line

(Also: "The most electronegative element is _____" – pipeline leaves alone,
        _____ stays as plain text, never enters math mode)
BUCKETS: [H]
```

Bucket I — code_snippet

```
INPUT  : Use `npm install` to install.
STAGE 4: classifier sees backticks → routes through text
BUCKETS: [G, I]
OUTPUT : (unchanged – rendered as plain text, no math parsing attempted)

INPUT  : ```python\nprint(1)\n```
STAGE 0: Triple-backtick fence → never enters math pipeline
BUCKETS: [I]
OUTPUT : Code block intact for downstream code highlighter
```

Bucket J — currency_text

```
INPUT : The book costs $5 and the pen costs $3.
STAGE 3: Two $ characters – segmenter creates 1 math segment "5 and the pen costs "
STAGE 4: CurrencySignal triggers (looks like prose-with-dollars, no math command)
        confidence drops below threshold
STAGE 5: Segment demoted to text
BUCKETS: [G, J]
OUTPUT : (unchanged – KaTeX never sees this as math)
```

Bucket K — nested_parser_corruption

```
INPUT :  $\frac{1}{\text{\sqrt{3}}}$ 
STAGE 5: Validator OK (braces balanced)
        KaTeX with strict:'ignore' renders best-effort
BUCKETS: [K]
OUTPUT :  $\frac{1}{\text{\sqrt{3}}}$ 
        (KaTeX produces some visual output; strict-mode users would see warning)
```

Bucket L — multiline_ai_solution

```
INPUT : Step 1: solve.\nStep 2: verify. ← literal \n from JSON
FAMILY : ai_solution
STAGE 1: LiteralEscapeRepairer in Tier 1 prose
        ✓ literal_escape_to_whitespace
OUTPUT : Step 1: solve.<actual newline>Step 2: verify.
BUCKETS: [L]
```

Bucket Z — clean

```
INPUT :  $\frac{1}{2} + \alpha$ 
STAGE 3: Segmenter → 1 math segment
STAGE 4: Tier 1 math no-op, Tier 2 not triggered (no corruption)
STAGE 5: Validator OK
BUCKETS: [Z]
OUTPUT : (unchanged – already correct)
```

7. SOLID compliance — verified by tests

The architecture is mechanically verified to satisfy SOLID principles
(`tests_solid/test_solid_principles.py`):

Principle	How enforced	Verified by
S — Single Responsibility	Each class implements one interface; segmenters segment, validators validate, signals detect	<code>TestSingleResponsibility</code>
O — Open/Closed	Adding a repairer/signal/validator extends behavior via the builder, no edits to existing code	<code>TestOpenClosed</code> — adds emoji-stripper at runtime
L — Liskov Substitution	Any class implementing the protocol is a valid drop-in	<code>TestLiskovSubstitution</code> — substitutes <code>StubSegmenter</code>
I — Interface Segregation	Every <code>I*</code> interface defines one method	<code>TestInterfaceSegregation</code>
D — Dependency Inversion	Pipeline orchestrator imports only from <code>core</code>	<code>TestDependencyInversion</code> — greps <code>pipeline.py</code> source

8. CLI surface

```
# Heal a single string
python -m app.autoheal "\alpha + eta = -\frac{1}{6}`$"
python -m app.autoheal --diagnose "<broken latex>" # show error positions
python -m app.autoheal --interactive # REPL
python -m app.autoheal --quiet "..." # pipe-able

# Heal a dataset
python -m app.run_pipeline --input file.csv --outdir out/
python -m app.run_pipeline --input file.jsonl --outdir out/ --limit 1000

# Build inspector data
python -m app.build_inspector --input file.jsonl --per-stratum 40 --max-rows 1500

# Serve inspector
python -m http.server 8765
start http://127.0.0.1:8765/web/inspector/index.html

# Tests
python -m unittest tests_solid.test_correct_latex tests_solid.test_broken_latex
tests_solid.test_solid_principles tests_solid.test_error_analyzer
python -m unittest tests.test_pipeline
python _e2e_autoheal.py
python _e2e_idempotence.py
python _validate_synth.py
node _e2e_js.js
node _e2e_user_input.js
```

9. Production-shape numbers (verified end-to-end)

Surface	Items	Time	Errors	Fallback
Unit tests (SOLID)	60	<0.01s	0	—
Unit tests (original)	28	<0.01s	0	—
E2E autoheal cases	15	<0.1s	0	0
E2E JS port cases	10	<1s	0	0
Synthetic dataset	1,010	<1s	0	0
<code>golden_test_set.csv</code>	500	0.09s	0	0
<code>confirmed_broken.csv</code>	1,500	0.20s	0	0
<code>confirmed_good.csv</code>	1,500	0.34s	0	0
<code>tricky_examples.csv</code>	500	0.04s	0	0
<code>labeled_sample.csv</code>	1,746	0.30s	0	0
<code>classified_candidates.jsonl</code> (master)	467,886	53.6s	0	0
<code>all_candidates.jsonl</code> (master)	467,886	53.1s	0	0
TOTAL	942,641	~109s	0	0

Idempotence verified on 5,000 random real rows (0 non-idempotent).

10. The five fixes that took fallback from 87 → 0

Case	Description	Fix mechanism	Rows recovered
1	HTML in question body (e.g. <code><table></code>) was treated as fallback	New <code>_run_html_aware</code> splits on tags, preserves HTML, renders math inside	69
2	Empty <code>\$\$</code> pairs from truncated AI prose	Segmenter coalesces empty math segments back into text	12
3	<code>\text{_____}</code> fill-in-the-blank crashed KaTeX	<code>FillBlankInTextRepairer</code> rewrites to <code>\rule{Nem}{0.4pt}</code>	4
4	Extra <code>}</code> indicating lost <code>\frac{</code> prefix	<code>MissingFracPrefixRepairer</code> rewrites <code>X}{Y}</code> as <code>\frac{X}{Y}</code>	2
5	Math segment emptied by Tier 1 repair (<code>\$\$</code> etc.)	Demote to TEXT containing original delimited form	2
Total			87

Net: every one of the 467,886 production rows now renders safely.

This document is a complete reference for the implementation. For source-level specifics, jump to the file paths listed in Section 5.
